

Question **1**

Correct

Marked out of
1.00

🚩 Flag question

Given a string, **s**, consisting of alphabets and digits, find the frequency of each digit in the given string.

Input Format

The first line contains a string, **num** which is the given number.

Constraints

$$1 \leq \text{len}(\text{num}) \leq 1000$$

All the elements of num are made of English alphabets and digits.

Output Format

Print ten space-separated integers in a single line denoting the frequency of each digit from **0** to **9**.

```

1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     char num[1000];
6     scanf("%s", num);
7     int freq[10]={0,0,0,0,0,0,0,0,0,0};
8     int temp;
9     for(int i=0;num[i]!='\0';i++)
10    {
11        temp= num[i]-'0';
12        if(temp<=9 && temp>=0)
13        {
14            freq[temp]++;
15        }
16    }
17    for(int i=0; i<10;i++)
18        printf("%d ",freq[i]);
19    return 0;
20 }
21
22

```

	Input	Expected	Got	
✓	a11472o5t6	0 2 1 0 1 1 1 1 0 0	0 2 1 0 1 1 1 1 0 0	✓
✓	lw4n88j12n1	0 2 1 0 1 0 0 0 2 0	0 2 1 0 1 0 0 0 2 0	✓
✓	1v88886l256338ar0ekk	1 1 1 2 0 1 2 0 5 0	1 1 1 2 0 1 2 0 5 0	✓

Passed all tests! ✓

Question **2**

Correct

Marked out of
1.00

🚩 Flag question

Today, Monk went for a walk in a garden. There are many trees in the garden and each tree has an English alphabet on it. While Monk was walking, he noticed that all trees with vowels on it are not in good state. He decided to take care of them. So, he asked you to tell him the count of such trees in the garden.

Note: The following letters are vowels: 'A', 'E', 'I', 'O', 'U', 'a', 'e', 'i', 'o' and 'u'.

Input:

The first line consists of an integer T denoting the number of test cases.

Each test case consists of only one string, each character of string denoting the alphabet (may be lowercase or uppercase) on a tree in the garden.

Output:

For each test case, print the count in a new line.

```

1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     int t;
6     char vowy[] = "aeiouAEIOU";
7     scanf("%d", &t);
8     while(t--)
9     {
10         char str[1000];
11
12         scanf("%s", str);
13         int vo=0;
14         for(int i=0; str[i]!='\0'; i++)
15         {
16
17             if(strchr(vowy, str[i])) vo++;
18         }
19         printf("%d\n", vo);
20     }
21     return 0;
22 }

```

	Input	Expected	Got	
✓	2 nBBZLaosnm JHkIsnZtTL	2 1	2 1	✓
✓	2 nBBZLaosnm JHkIsnZtTL	2 1	2 1	✓

Question **3**

Correct

Marked out of
1.00

🚩 Flag question

Given a sentence, s , print each word of the sentence in a new line.

Input Format

The first and only line contains a sentence, s .

Constraints

$$1 \leq \text{len}(s) \leq 1000$$

Output Format

Print each word of the sentence in a new line.

Sample Input 0

This is C

```
1 | #include <stdio.h>
```



```
1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     char sent[1000];
6     scanf("%[^\n]", sent);
7     char *word;
8
9     word=strtok(sent, " ");
10    while(word!=NULL)
11    {
12        printf("%s\n",word);
13        word=strtok(NULL, " ");
14    }
15    return 0;
16 }
```

	Input	Expected	Got	
✓	This is C	This is C	This is C	✓
✓	Learning C is fun	Learning C is fun	Learning C is fun	✓

Passed all tests! ✓

Question **4**

Correct

Marked out of
1.00

 [Flag question](#)

Input Format

You are given two strings, ***a*** and ***b***, separated by a new line. Each string will consist of lower case Latin characters ('a'-'z').

Output Format

In the first line print two space-separated integers, representing the length of ***a*** and ***b*** respectively.

In the second line print the string produced by concatenating ***a*** and ***b*** (***a + b***).

In the third line print two strings separated by a space, ***a'*** and ***b'***. ***a'*** and ***b'*** are the same as ***a*** and ***b***, respectively, except that their first characters are swapped.

Sample Input

abcd

ef

Sample Output

4 2

abcdef

ebcd af


```
1 #include <stdio.h>
2 #include <string.h>
3 int main()
4 {
5     char a[10], b[10];
6     scanf("%s", a);
7     scanf("%s", b);
8     printf("%zu %zu", (strlen(a)), strlen(b));
9     printf("\n");
10
11     printf("%s%s", a,b);
12     printf("\n");
13     char t;
14     t=a[0];
15     a[0]=b[0];
16     b[0]=t;
17     printf("%s %s", a, b);
18     return 0;
19 }
```

	Input	Expected	Got	
✓	abcd ef	4 2 abcdef ebcd af	4 2 abcdef ebcd af	✓

Passed all tests! ✓